ОБРАЗОВАТЕЛЬНАЯ ОРГАНИЗАЦИЯ: _____

Кафедра / отделение: _____

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

РАЗРАБОТКА АЛГОРИТМОВ ГЕНЕРАЦИИ ПСЕВДОСЛУЧАЙНЫХ ПОСЛЕДОВАТЕЛЬНОСТЕЙ

Вариант 12

Дисциплина: _____

Выполнил(а): _____

Группа: _____

Проверил(а): _____

Город: _____ 2026 г.

1. Постановка задачи

Цель работы — изучить генерацию псевдослучайных последовательностей с помощью srand() и rand(), оценку равномерности по частотам и применение генератора в игровой программе.

Задача 1, вариант 12. Сформировать массив из N = 750 чисел в интервале [100; 110]. Разбить его на 10 равных интервалов длиной 1, подсчитать частоты попаданий и построить горизонтальную гистограмму. В программе используется графическая библиотека Cairo; окно выводится через X11, график также сохраняется в PNG.

Задача 2.4. Реализовать «Крестики-нолики» на прямоугольном поле. Побеждает игрок, первым выстроивший пять одинаковых символов подряд по горизонтали, вертикали или диагонали. Выбрано поле 10 строк × 15 столбцов. Реализованы два режима: два человека и человек против компьютера. X ходит первым; компьютер играет за O и выбирает случайную свободную клетку. Заполненное поле без победителя означает ничью.

Среда реализации: язык C, GCC, Linux, Cairo и X11. Функции srand(), rand() и time() доступны в C; перевод программы на C++ для выполнения условия не требуется.

2. Алгоритм решения задачи

2.1. Генерация массива и построение гистограммы

1. Инициализировать генератор вызовом srand((unsigned int)time(NULL)). Для воспроизводимого контрольного запуска передать --seed 12.

2. Для каждого из 750 элементов вычислить $x = 100 + 10 \cdot \text{rand}() / \text{RAND_MAX}$, выполняя деление в типе double. Возможные значения включают обе границы.

3. Обнулить 10 счётчиков. Для каждого x вычислить $j = \text{floor}((x - 100) / 10 \cdot 10)$. Если j = 10, заменить его на 9, чтобы значение 110 попало в последний интервал. Увеличить соответствующий счётчик.

4. Интервалы имеют вид [100; 101), [101; 102), …, [109; 110]. Каждое число учитывается ровно один раз. Контроль: сумма частот должна равняться 750.

5. Построить горизонтальные столбцы: их длина пропорциональна частоте. Подписать границы интервалов и количества чисел. Отметить красной пунктирной линией ожидаемую частоту N / 10 = 75. Сохранить histogram.png и открыть окно графика.

6. Сравнить частоты с 75. Небольшие различия естественны для конечной случайной выборки. Гистограмма даёт визуальную оценку, но сама по себе не доказывает равномерность или независимость последовательности.

2.2. Игра «Крестики-нолики»

1. Создать пустой массив поля 10 × 15; запросить режим игры, назначить X первым игроком.

2. Для человека прочитать строку и столбец с нумерацией от 1. Проверить формат ввода, границы поля и свободна ли клетка. При ошибке повторить запрос, сохранив очередь игрока. Команда 0 0 завершает игру.

3. Для компьютера составить список свободных клеток, выбрать элемент по индексу rand() % count и поставить O. Такой компьютер делает допустимые случайные ходы, но не использует выигрышную стратегию.

4. После хода вывести поле. Проверить четыре направления: (0, 1), (1, 0), (1, 1), (1, −1). В каждом направлении считать непрерывную цепочку одинаковых символов в обе стороны от последнего хода, включая сам ход.

5. Если длина цепочки не меньше пяти, объявить победителя. Иначе при заполненном поле объявить ничью. Если игра продолжается, сменить игрока и повторить шаг 2 или 3.

2.3. Сборка и запуск

```
make
./histogram
```

В меню выбрать 1 для гистограммы или 2 для игры. При выборе игры указать 1 для двух игроков или 2 для игры против компьютера.

Прямые команды запуска:

```
./histogram --histogram
./histogram --png
./histogram --game
make test
```

Команда --png создаёт изображение без открытия окна. Закрыть окно гистограммы можно кнопкой закрытия или клавишей Escape. Альтернативная сборка без Makefile:

```
gcc main.c -o histogram $(pkg-config --cflags --libs cairo x11)
```

Обычной команды gcc main.c недостаточно: требуется указать заголовки и библиотеки Cairo/X11. Они уже доступны в использованной среде.

3. Контрольный тест

3.1. Гистограмма

```
./histogram --png --seed 12
```

Начальное значение генератора — 12. В одной и той же реализации rand() такой запуск воспроизводится. В другой стандартной библиотеке конкретная последовательность может отличаться.

Интервал	Ожидается	Получено
[100; 101)	75	90
[101; 102)	75	78
[102; 103)	75	76
[103; 104)	75	75
[104; 105)	75	71
[105; 106)	75	65
[106; 107)	75	81
[107; 108)	75	68
[108; 109)	75	75
[109; 110]	75	71
Итого	750	750

Результат: минимум частоты — 65, максимум — 90. Максимальное абсолютное отклонение от 75 составляет 15 чисел, или 20% ожидаемой частоты. Все значения лежат в [100; 110], пропущенных и повторно учтённых элементов нет. На гистограмме нет резко выделяющихся пустых или переполненных интервалов; результат согласуется с приблизительно равномерным распределением этой выборки.

3.2. Контрольная партия

Режим: два игрока. Поочерёдно вводятся следующие координаты (строка, столбец):

Ход	Игрок	Координаты
1	X	1 1
2	O	2 1
3	X	1 2
4	O	2 2
5	X	1 3
6	O	2 3
7	X	1 4
8	O	2 4
9	X	1 5

Ожидаемый результат: после девятого хода в первой строке стоят пять X подряд, программа объявляет победу X. Фактический результат: «Победил X: пять подряд!». Результаты совпали.

```
./histogram --game < report/game_input.txt
```

3.3. Дополнительные проверки

Проверка	Ожидаемый и полученный результат
Границы интервалов	100 → первый; 101 → второй; 110 → последний
Четыре символа подряд	Победы нет
Пять в четырёх направлениях	Победа распознаётся в каждом случае
Шесть подряд у края	Победа распознаётся
Занятая клетка и выход за поле	Ход отклонён, очередь сохранена
Неверный формат ввода	Повторный запрос координат
Единственная свободная клетка	Компьютер занимает её
Полное поле без пяти подряд	Ничья
Контрольная партия из 9 ходов	Победа X

Проверки выполнены программами tests.c и tests_integration.py. Команда make test завершилась успешно. Сборка выполнена с -Wall -Wextra -pedantic; для модульных тестов включён -Werror.

4. Скриншоты работы программы

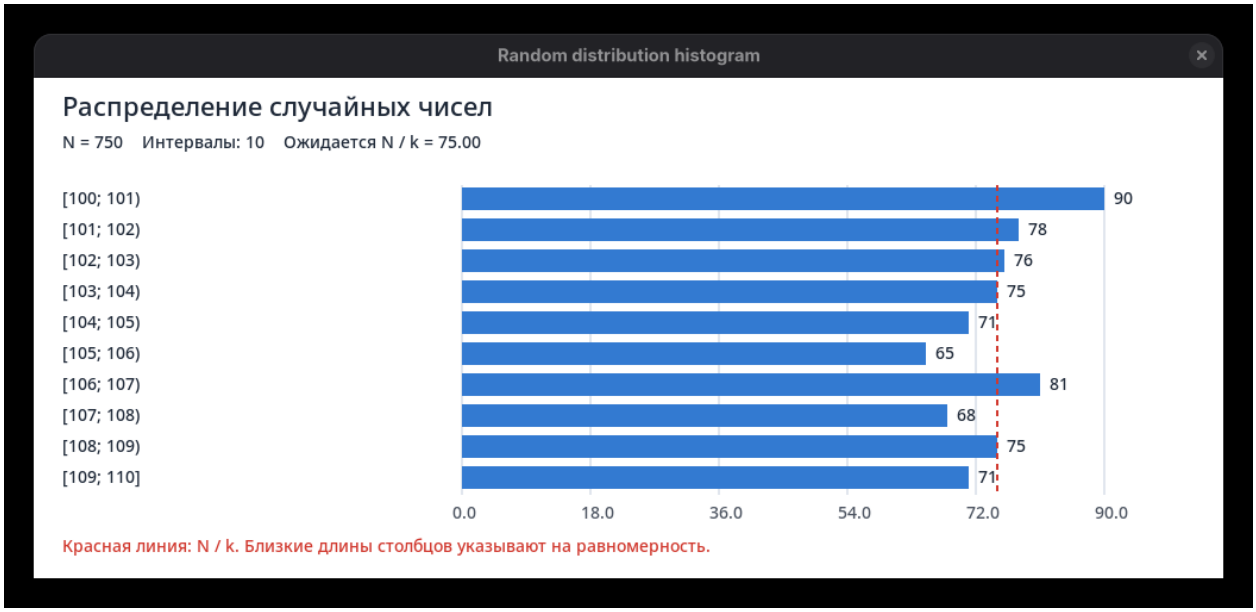


Рисунок 1 — окно гистограммы, N = 750, [100; 110], seed = 12.

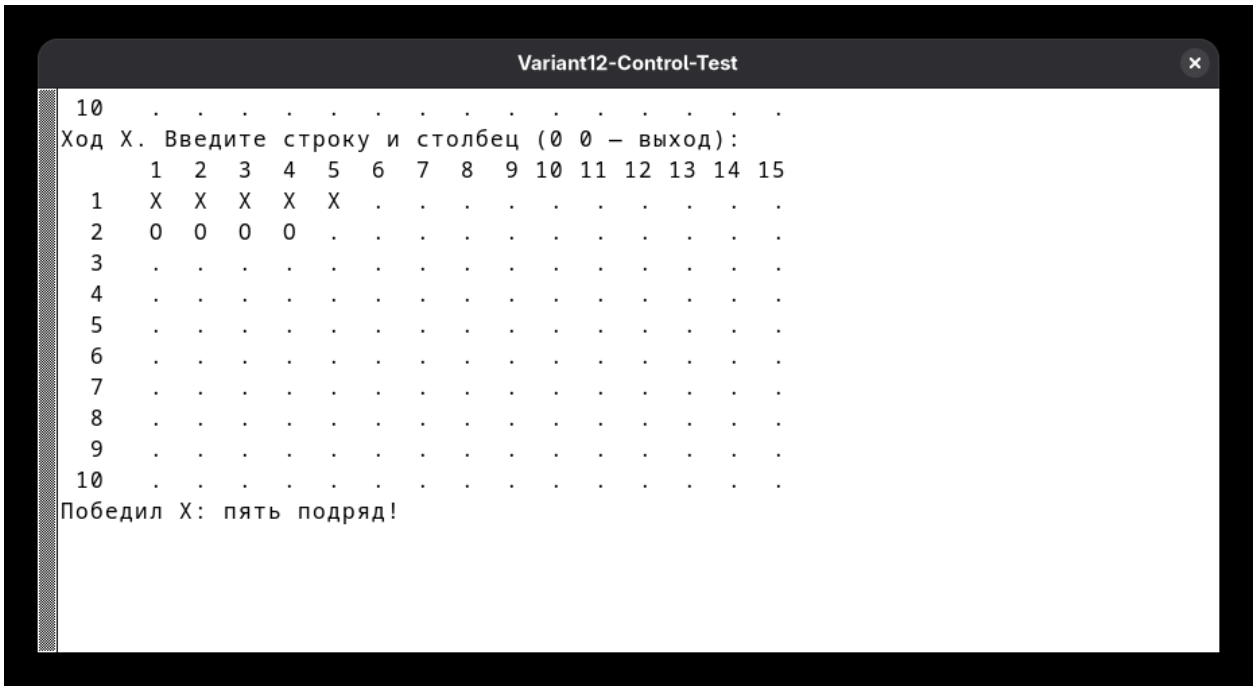


Рисунок 2 — итог контрольной партии: пять X в первой строке и сообщение о победе.

Изображения получены захватом окон запущенной программы. Для второго скриншота в терминале показаны последние 14 строк фактического вывода контрольной партии. Полный вывод сохранён в report/game_test.txt.

5. Выводы по работе

Разработана программа для варианта 12, создающая массив из 750 псевдослучайных вещественных чисел в интервале [100; 110]. Построена графическая горизонтальная гистограмма по 10 равным интервалам. Сумма частот в контрольном запуске составила 750, значения частот — от 65 до 90 при ожидаемой частоте 75.

Изучено влияние начального значения генератора: фиксированное значение позволяет повторить опыт в той же среде, а значение таймера меняет последовательность между запусками, начатыми в разные секунды. Использование таймера не превращает детерминированный rand() в источник истинной случайности.

Реализована игра с пятью символами подряд на поле 10 × 15. Проверены горизонтальные, вертикальные и диагональные победы, корректность ходов и ничья. Случайный выбор свободной клетки демонстрирует применение ГПСЧ в поведении компьютера.

Визуальное сравнение частот удобно для предварительной оценки равномерности. Для более строгого исследования качества генератора потребовались бы дополнительные статистические испытания и проверка зависимости между соседними значениями.

6. Листинг программы

Файл main.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <cairo.h>
#include <cairo-xlib.h>
#include <X11/Xutil.h>


/* Сборка: gcc main.c -o histogram $(pkg-config --cflags --libs cairo x11) */
#define WINDOW_WIDTH 1000
#define N 750
#define BINS 10
#define A 100.0
#define B 110.0
#define ROWS 10
#define COLS 15


static void label(cairo_t *cr, double x, double y, const char *text) {
  cairo_move_to(cr, x, y);
  cairo_show_text(cr, text);
}


static void draw_histogram(cairo_t *cr, int width, int height,
                           const int *counts, int n, int k, double a, double b) {
  char text[256];
  double expected = (double)n / k;
  int maximum = 1;
  for (int i = 0; i < k; i++) {
    if (counts[i] > maximum) maximum = counts[i];
  }


  /* Используем одинаковые координаты для окна и PNG. */
  cairo_save(cr);
  cairo_scale(cr, (double)width / WINDOW_WIDTH,
              (double)height / (160 + 26 * k));
  cairo_set_source_rgb(cr, 1, 1, 1);
  cairo_paint(cr);
  cairo_select_font_face(cr, "Sans", CAIRO_FONT_SLANT_NORMAL,
                         CAIRO_FONT_WEIGHT_NORMAL);
  cairo_set_source_rgb(cr, 0.12, 0.16, 0.22);
  cairo_set_font_size(cr, 22);
  label(cr, 24, 32, "Распределение случайных чисел");
  cairo_set_font_size(cr, 14);
  snprintf(text, sizeof(text), "N = %d    Интервалы: %d    Ожидается N / k = %.2f", n, k,
expected);
  label(cr, 24, 59, text);


  const double left = 360, top = 90, plot_width = 540;
  double scale = plot_width / maximum;
  for (int tick = 0; tick <= 5; tick++) {
    double x = left + plot_width * tick / 5;
    cairo_set_source_rgb(cr, 0.88, 0.90, 0.93);
    cairo_move_to(cr, x, top);
    cairo_line_to(cr, x, top + 26 * k);
    cairo_stroke(cr);
    cairo_set_source_rgb(cr, 0.30, 0.33, 0.38);
    snprintf(text, sizeof(text), "%.1f", (double)maximum * tick / 5);
    label(cr, x - 8, top + 26 * k + 20, text);
  }


  for (int i = 0; i < k; i++) {
    double lo = a + (b - a) * ((double)i / k);
    double hi = i == k - 1 ? b : a + (b - a) * ((double)(i + 1) / k);
    double y = top + 26 * i;
    cairo_set_source_rgb(cr, 0.12, 0.16, 0.22);
    snprintf(text, sizeof(text), "[%.7g; %.7g%s", lo, hi, i == k - 1 ? "]" : ")");
    label(cr, 24, y + 16, text);
    cairo_set_source_rgb(cr, 0.20, 0.48, 0.82);
    cairo_rectangle(cr, left, y + 2, counts[i] * scale, 19);
    cairo_fill(cr);
```

```
    cairo_set_source_rgb(cr, 0.12, 0.16, 0.22);
    snprintf(text, sizeof(text), "%d", counts[i]);
    label(cr, left + counts[i] * scale + 8, y + 16, text);
  }

  /* Красная пунктирная линия — ожидаемое число попаданий. */
  double dash[] = {5, 4};
  cairo_set_source_rgb(cr, 0.82, 0.22, 0.18);
  cairo_set_line_width(cr, 2);
  cairo_set_dash(cr, dash, 2, 0);
  cairo_move_to(cr, left + expected * scale, top);
  cairo_line_to(cr, left + expected * scale, top + 26 * k);
  cairo_stroke(cr);
  label(cr, 24, top + 26 * k + 48, "Красная линия: N / k. Близкие длины столбцов указывают
на равномерность.");
  cairo_restore(cr);
}

static int show_window(const int *counts, int n, int k, double a, double b) {
  Display *display = XOpenDisplay(NULL);
  if (display == NULL) {
    fprintf(stderr, "Не удалось открыть окно X11. График сохранён в histogram.png.\n");
    return 1;
  }
  int screen = DefaultScreen(display);
  int width = WINDOW_WIDTH, height = 160 + 26 * k;
  Window window = XCreateSimpleWindow(display, RootWindow(display, screen),
      0, 0, width, height, 0, BlackPixel(display, screen), WhitePixel(display, screen));
  XStoreName(display, window, "Random distribution histogram");
  XSelectInput(display, window, ExposureMask | StructureNotifyMask | KeyPressMask);
  Atom close_message = XInternAtom(display, "WM_DELETE_WINDOW", False);
  Atom protocols = XInternAtom(display, "WM_PROTOCOLS", False);
  XSetWMProtocols(display, window, &close_message, 1);
  XMapWindow(display, window);
  cairo_surface_t *surface = cairo_xlib_surface_create(
      display, window, DefaultVisual(display, screen), width, height);
  cairo_t *cr = cairo_create(surface);

  for (;;) {
    XEvent event;
    XNextEvent(display, &event);
    if ((event.type == ClientMessage && event.xclient.message_type == protocols &&
         (Atom)event.xclient.data.l[0] == close_message) ||
        (event.type == KeyPress && XLookupKeysym(&event.xkey, 0) == XK_Escape)) break;
    if (event.type == ConfigureNotify) {
      width = event.xconfigure.width;
      height = event.xconfigure.height;
      cairo_xlib_surface_set_size(surface, width, height);
    }
    if (event.type == Expose || event.type == ConfigureNotify) {
      draw_histogram(cr, width, height, counts, n, k, a, b);
      cairo_surface_flush(surface);
      XFlush(display);
    }
  }
  int failed = cairo_status(cr) != CAIRO_STATUS_SUCCESS;
  cairo_destroy(cr);
  cairo_surface_destroy(surface);
  XDestroyWindow(display, window);
  XCloseDisplay(display);
  return failed;
}

/* b включается только в последний интервал. */
static int interval_index(double value) {
  int index = (int)((value - A) / (B - A) * BINS);
  return index >= BINS ? BINS - 1 : index;
}

static void generate_numbers(double numbers[N], int counts[BINS]) {
  for (int i = 0; i < BINS; i++) counts[i] = 0;
  for (int i = 0; i < N; i++) {
    numbers[i] = A + (B - A) * ((double)rand() / RAND_MAX);
    counts[interval_index(numbers[i])]++;
  }
```

```
}

static int run_histogram(int png_only) {
  double numbers[N];
  int counts[BINS];
  generate_numbers(numbers, counts);
  printf("Вариант 12: N = %d, [%.0f; %.0f], интервалов: %d\n", N, A, B, BINS);
  for (int i = 0; i < BINS; i++) {
    printf("[%.0f; %.0f%s %d\n", A + i, A + i + 1,
           i == BINS - 1 ? "]" : ")", counts[i]);
  }
  printf("Ожидаемое количество в каждом интервале: %.0f\n", (double)N / BINS);

  cairo_surface_t *png = cairo_image_surface_create(
      CAIRO_FORMAT_ARGB32, WINDOW_WIDTH, 160 + 26 * BINS);
  cairo_t *cr = cairo_create(png);
  draw_histogram(cr, WINDOW_WIDTH, 160 + 26 * BINS, counts, N, BINS, A, B);
  cairo_status_t status = cairo_status(cr);
  if (status == CAIRO_STATUS_SUCCESS)
    status = cairo_surface_write_to_png(png, "histogram.png");
  cairo_destroy(cr);
  cairo_surface_destroy(png);
  if (status != CAIRO_STATUS_SUCCESS) {
    fprintf(stderr, "Ошибка сохранения: %s\n", cairo_status_to_string(status));
    return 1;
  }
  printf("График сохранён в histogram.png.\n");
  return png_only ? 0 : show_window(counts, N, BINS, A, B);
}

/* Нулевой элемент означает свободную клетку. */
static int put_mark(char board[ROWS][COLS], int row, int col, char mark) {
  if (row < 0 || row >= ROWS || col < 0 || col >= COLS || board[row][col])
    return 0;
  board[row][col] = mark;
  return 1;
}

static int has_five(char board[ROWS][COLS], int row, int col) {
  const int directions[4][2] = {{0, 1}, {1, 0}, {1, 1}, {1, -1}};
  char mark = board[row][col];
  if (!mark) return 0;
  for (int d = 0; d < 4; d++) {
    int count = 1;
    for (int sign = -1; sign <= 1; sign += 2) {
      int r = row + sign * directions[d][0];
      int c = col + sign * directions[d][1];
      while (r >= 0 && r < ROWS && c >= 0 && c < COLS && board[r][c] == mark) {
        count++;
        r += sign * directions[d][0];
        c += sign * directions[d][1];
      }
    }
    if (count >= 5) return 1;
  }
  return 0;
}

/* Выбираем случайную свободную клетку. */
static int computer_move(char board[ROWS][COLS], int *row, int *col) {
  int free_cells[ROWS * COLS], count = 0;
  for (int r = 0; r < ROWS; r++)
    for (int c = 0; c < COLS; c++)
      if (!board[r][c]) free_cells[count++] = r * COLS + c;
  if (count == 0) return 0;
  int cell = free_cells[rand() % count];
  *row = cell / COLS;
  *col = cell % COLS;
  return put_mark(board, *row, *col, 'O');
}

static void print_board(char board[ROWS][COLS]) {
  printf("\n    ");
  for (int c = 1; c <= COLS; c++) printf("%3d", c);
  putchar('\n');
```

```
  for (int r = 0; r < ROWS; r++) {
    printf("%3d ", r + 1);
    for (int c = 0; c < COLS; c++)
      printf("%3c", board[r][c] ? board[r][c] : '.');
    putchar('\n');
  }
}

static int run_game(void) {
  char board[ROWS][COLS] = {{0}};
  char line[128], extra;
  int mode;
  printf("Крестики-нолики: пять подряд. Поле %d x %d.\n", ROWS, COLS);
  for (;;) {
    printf("Режим: 1 — два игрока, 2 — против компьютера: ");
    fflush(stdout);
    if (!fgets(line, sizeof(line), stdin)) return 0;
    if (sscanf(line, "%d %c", &mode, &extra) == 1 && (mode == 1 || mode == 2)) break;
    printf("Введите 1 или 2.\n");
  }
  char turn = 'X';
  int moves = 0;
  print_board(board);
  while (moves < ROWS * COLS) {
    int row, col;
    if (mode == 2 && turn == 'O') {
      if (!computer_move(board, &row, &col)) break;
      printf("Компьютер O: %d %d\n", row + 1, col + 1);
    } else {
      printf("Ход %c. Введите строку и столбец (0 0 — выход): ", turn);
      fflush(stdout);
      if (!fgets(line, sizeof(line), stdin)) return 0;
      if (sscanf(line, "%d %d %c", &row, &col, &extra) != 2) {
        printf("Нужно ввести два целых числа.\n");
        continue;
      }
      if (row == 0 && col == 0) return 0;
      if (row < 1 || row > ROWS || col < 1 || col > COLS ||
          !put_mark(board, row - 1, col - 1, turn)) {
        printf("Недопустимый ход: клетка занята или находится вне поля.\n");
        continue;
      }
      row--;
      col--;
    }
    moves++;
    print_board(board);
    if (has_five(board, row, col)) {
      printf("Победил %c: пять подряд!\n", turn);
      return 0;
    }
    turn = turn == 'X' ? 'O' : 'X';
  }
  printf("Ничья: свободных клеток нет.\n");
  return 0;
}

int main(int argc, char **argv) {
  int task = 0, png_only = 0;
  unsigned int seed = (unsigned int)time(NULL);
  for (int i = 1; i < argc; i++) {
    if (strcmp(argv[i], "--png") == 0) { task = 1; png_only = 1; }
    else if (strcmp(argv[i], "--histogram") == 0) task = 1;
    else if (strcmp(argv[i], "--game") == 0) task = 2;
    else if (strcmp(argv[i], "--seed") == 0 && i + 1 < argc) {
      char extra;
      if (sscanf(argv[++i], "%u %c", &seed, &extra) != 1) return 1;
    } else {
      fprintf(stderr, "Использование: %s [--histogram | --png | --game] [--seed число]\n",
argv[0]);
      return 1;
    }
  }
  srand(seed);
  if (task == 0) {
    char line[128], extra;
```

```
    printf("Вариант 12\n1 — гистограмма\n2 — крестики-нолики\nВыберите задачу: ");
    fflush(stdout);
    if (!fgets(line, sizeof(line), stdin) ||
        sscanf(line, "%d %c", &task, &extra) != 1 || (task != 1 && task != 2)) {
      fprintf(stderr, "Нужно выбрать задачу 1 или 2.\n");
      return 1;
    }
  }
  return task == 1 ? run_histogram(png_only) : run_game();
}
```

6.1. Файл сборки Makefile

```
CC = gcc
CFLAGS = -std=c11 -Wall -Wextra -pedantic
CPPFLAGS += $(shell pkg-config --cflags cairo x11)
LDLIBS += $(shell pkg-config --libs cairo x11)

.PHONY: all test
all: histogram

histogram: main.c
	$(CC) $(CPPFLAGS) $(CFLAGS) $< -o $@ $(LDFLAGS) $(LDLIBS)

tests: tests.c main.c
	$(CC) $(CPPFLAGS) $(CFLAGS) -Werror tests.c -o $@ $(LDFLAGS) $(LDLIBS)

test: histogram tests
	./tests
	python3 tests_integration.py
```